Day 2 — Java Programming Fundamentals

Simple Questions, Answers, Setup Steps, and Practice Programs

1. What is Java?

Java is a high-level, object-oriented programming language. It is widely used to build backend applications, enterprise software, Android applications, and web services. Java code is compiled into bytecode, which runs on the Java Virtual Machine (JVM).

2. What are JDK, JRE, and JVM?

Term	Meaning	Purpose
JDK	Java Development Kit	Tools to develop, compile, and run Java programs. Includes the compiler and runtime components.
JRE	Java Runtime Environment	Components needed to run Java applications. In modern Java distributions, a separate JRE may not be provided.
JVM	Java Virtual Machine	Executes Java bytecode.

Remember: JDK is used to develop Java programs; JVM runs Java bytecode.

3. How do you set up the Java environment?

1. Install a JDK (for example, JDK 21) from a trusted Java distribution.

2. Open Command Prompt or a terminal.

3. Run **java -version** to check the Java runtime.

4. Run **javac -version** to check the Java compiler.

5. Create a file named **Main.java**, write a Java program, then compile and run it.

```
javac Main.java
java Main
```

4. What are variables and data types?

A variable is a named place used to store a value. A data type specifies what kind of value a variable can hold.

```
int age = 22;             // whole number
double price = 99.50;     // decimal number
char grade = 'A';         // one character
boolean isJavaEasy = true; // true or false
String name = "Amit";     // text
```

Common primitive data types include byte, short, int, long, float, double, char, and boolean. String is a class used to store text.

5. What are operators?

Operators perform actions on values. Arithmetic operators include +, -, *, /, and %. Comparison operators include ==, !=, >, <, >=, and <=. Logical operators include && (AND), || (OR), and ! (NOT). Assignment operators include =, +=, and -=.

```
int a = 10, b = 3;
System.out.println(a + b);  // 13
System.out.println(a % b);  // 1
System.out.println(a > b);  // true
```

6. What are conditional statements?

Conditional statements allow a program to choose what to do based on a condition. Java provides if, else if, else, and switch.

```
int age = 18;
if (age >= 18) {
    System.out.println("Eligible to vote");
} else {
    System.out.println("Not eligible to vote");
}
```

7. What are loops?

Loops repeat a block of code. A for loop is useful when the number of repetitions is known. A while loop repeats while a condition is true. A do-while loop runs its body at least once.

```
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

8. How do you take input and display output?

System.out.println() displays output. Scanner can read input from the keyboard. Import java.util.Scanner before using Scanner.

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);
System.out.print("Enter your age: ");
int age = sc.nextInt();
System.out.println("Your age is " + age);
sc.close();
```

9. Practice Program: Check Even or Odd

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();

        if (n % 2 == 0) {
            System.out.println("Even");
        } else {
            System.out.println("Odd");
        }
        sc.close();
    }
}
```

10. Practice Program: Find the Largest of Three Numbers

```
int a = 12, b = 25, c = 9;

if (a >= b && a >= c) {
    System.out.println("Largest: " + a);
} else if (b >= a && b >= c) {
    System.out.println("Largest: " + b);
} else {
    System.out.println("Largest: " + c);
}
```

This example assumes the variables have already been declared inside main().

11. Practice Program: Calculate Factorial

```
int n = 5;
long factorial = 1;

for (int i = 1; i <= n; i++) {
    factorial *= i;
}
System.out.println("Factorial: " + factorial); // 120
```

Factorial of 5 is 5 × 4 × 3 × 2 × 1 = 120. This simple loop is intended for non-negative integers small enough to fit in long.

12. Practice Program: Check Whether a Number is Prime

```
int n = 17;
boolean isPrime = n >= 2;

for (int i = 2; i <= n / i; i++) {
    if (n % i == 0) {
        isPrime = false;
        break;
    }
}
System.out.println(isPrime ? "Prime" : "Not prime");
```

A prime number is greater than 1 and has exactly two positive factors: 1 and itself.

13. Practice Program: Reverse a Number

```
int n = 1234;
int reverse = 0;

while (n != 0) {
    int digit = n % 10;
    reverse = reverse * 10 + digit;
    n /= 10;
}
System.out.println("Reverse: " + reverse); // 4321
```

This example reverses a positive integer. Leading zeroes in the reversed result are not retained because the result is stored as an integer.

14. Practice Program: Print a Multiplication Table

```
int n = 5;

for (int i = 1; i <= 10; i++) {
    System.out.println(n + " x " + i + " = " + (n * i));
}
```

15. Basic Problem-Solving Steps

1. Understand the problem and the expected output.

2. Identify the input values and required calculation.

3. Break the solution into small steps or an algorithm.

4. Write the Java code using variables, operators, conditions, and loops.

5. Test normal cases and edge cases, then fix any errors.

Day 2 — Task Checklist

■ Install a JDK and verify java -version and javac -version.

■ Write and run a basic Java program.

■ Practise variables, data types, operators, conditions, loops, and keyboard input.

■ Complete the six practice programs: even/odd, largest of three, factorial, prime check, reverse a number, and multiplication table.

Quick Revision

JDK develops and compiles Java programs. **JVM** executes bytecode. **Variables** store values. **Operators** perform calculations or comparisons. **Conditions** make decisions. **Loops** repeat instructions. **Input** receives data, and **output** displays results.